UNIVERSIDADE ESTADUAL DE MARINGÁ
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA

Disciplina: DIN6920- Sistemas Distribuídos
Bacharelado em Ciência da Computação
Professor Dr. Alisson Renan Svaigen

Trabalho Prático

Implementação de um Serviço Distribuído de Quiz Online

Especificação Geral

Este trabalho tem como objetivo utilizar os conceitos teóricos de comunicação entre processos e balanceamento de carga em Sistemas Distribuídos para desenvolver um serviço de "Quiz Online". O serviço deve dar suporte para o acesso de *n* clientes por meio de 3 servidores distintos. Deve haver um balanceador de carga para distribuir as requisições aos servidores e, de maneira análoga, um SGBD que presta serviço de persistência dos dados aos servidores. A arquitetura geral do sistema a ser desenvolvido deve seguir o que é apresentado na Figura 1.

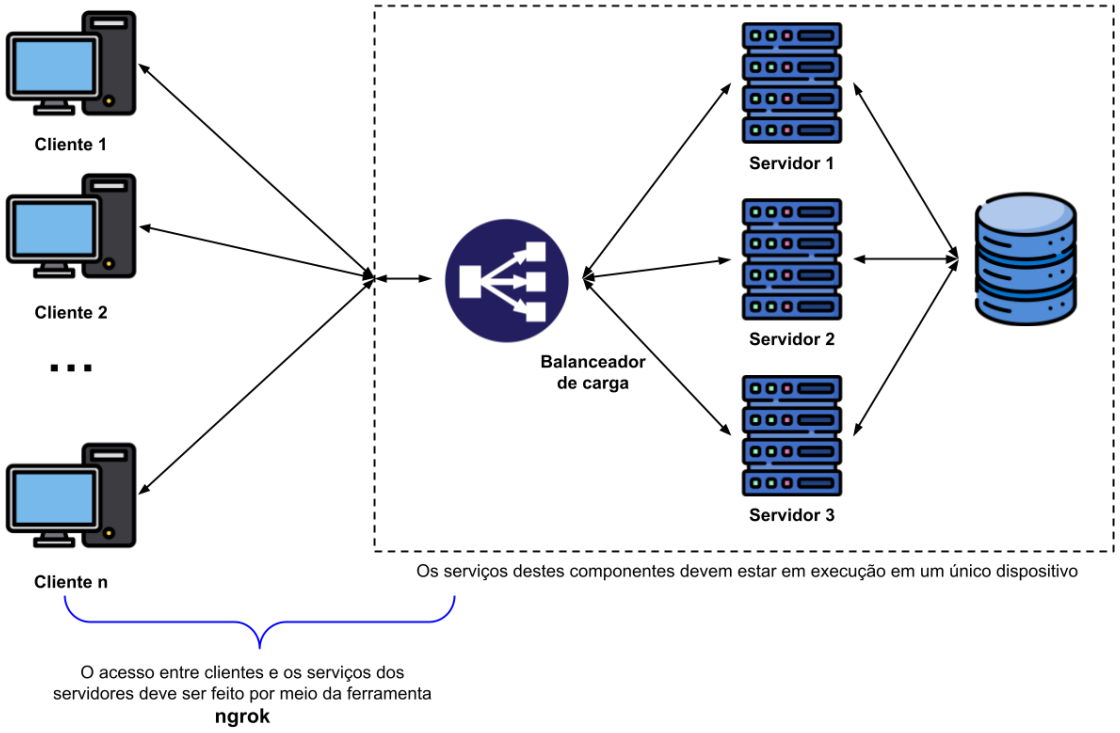


Figura 1: Arquitetura base de implementação do SD do trabalho prático

Devido às limitações de infraestrutura para execução do trabalho utilizando, no mínimo, 6 dispositivos computacionais distintos (ao menos 1 cliente, 1 balanceador, 3 servidores e 1 SGBD), a execução do sistema poderá ser realizada por meio de, no mínimo, dois dispositivos computacionais: ao menos 1 dispositivo executando as tarefas do cliente; e obrigatoriamente 1 dispositivo executando as tarefas do balanceador, dos servidores e do SGBD.

De modo geral, a arquitetura segue o padrão cliente x servidor, considerando um balanceador centralizado. No entanto, todos os 3 servidores devem executar localmente, no mesmo dispositivo que o balanceador de carga. Desse modo, o balanceador de carga irá distribuir as requisições para o mesmo dispositivo, de maneira local.

De modo a facilitar questões relacionadas ao roteamento, acesso e liberação de portas nos dispositivos, é **obrigatória** a utilização da ferramenta **ngrok**[1]. Esta ferramenta deve ser utilizada apenas como mecanismo de acesso externo ao balanceador. Não é necessário, nem desejável, utilizar uma instância do ngrok para cada servidor e para a comunicação com o SGBD.

Características do Quiz Online

O Quiz Online a ser desenvolvido deve ser inspirado no programa de TV "Show do Milhão", muito popular nos anos 2000 e que, recentemente, voltou a TV aberta nacional. O jogo consiste em responder uma série de perguntas com múltiplas alternativas, na qual o jogador tem à disposição um conjunto de ajudas para lhe auxiliar na empreitada.

Para este trabalho em específico, o Quiz deve ter, obrigatoriamente, as seguintes características:

- **Tema:** livre, exceto "Conhecimentos Gerais". Exemplos de temas válidos: Computação, Esportes, Copa do Mundo, K-pop, etc. (qualquer coisa diferente de "conhecimentos gerais").
- **Quantidade e formato de perguntas:**
    - **Válidas para o jogo:** 10 perguntas, divididas em 4 níveis: 3 perguntas fáceis, 3 médias, 3 difíceis e 1 pergunta final (que deve ser mais difícil que as demais). As perguntas que compõem um jogo devem ser sorteadas de um conjunto de perguntas do banco de dados;

[1] Disponível com plano gratuito em: https://ngrok.com/. Entenda mais sobre o **ngrok** em: https://medium.com/desenvolvendo-com-paixao/ngrok-do-localhost-para-o-mundo-5445ad08419

- **Possíveis de serem sorteadas para o jogo:** no mínimo, 12 perguntas para cada nível, totalizando 48 perguntas;
- **Formato**: cada pergunta deve ter 1 enunciado e 4 alternativas, sendo 3 **in**corretas e 1 correta.

- **Ajudas para o jogador:**
  - **1 "meio a meio"**: o sistema deve sortear 2 alternativas incorretas e eliminá-las, restando ao jogador escolher entre a alternativa correta e uma incorreta;
  - **1 pulo**: permite ao jogador pular uma pergunta sorteada. Essa ajuda pode ser utilizada após a ajuda de "meio a meio";
  - **1 "passe livre":** ao acionar essa ajuda, o jogador automaticamente acerta a pergunta. Não é dada pontuação bônus. Essa ajuda não pode ser utilizada para a pergunta final.
- **Pontuação:**
  - Para cada pergunta respondida corretamente, deve ser atribuída a seguinte pontuação:
    - **Nível "fácil":** 50 pontos;
    - **Nível "médio":** 100 pontos;
    - **Nível "difícil":** 300 pontos;
    - **Pergunta final:** 1000 pontos;
  - Para cada pergunta respondida corretamente, deve ainda ser dada uma pontuação bônus dada a seguinte fórmula:
    - `bônus = max(0, (60 - segundos)) * fator`
    - Na qual:
      - `segundos` representa quanto tempo (em segundos) o jogador demorou para responder a questão
      - `fator` é um valor inteiro dado de acordo com o nível da questão:
        - Nível "fácil": 1;
        - Nível "médio": 1.5;
        - Nível "difícil": 2;
        - Pergunta final: 10;

Implementação dos componentes do Sistema Distribuído

Os componentes, do ponto de vista de um Sistema Distribuído, são descritos a seguir, levando em consideração o que é apresentado na Figura 1.

Cliente: Interação

O cliente deverá fornecer uma interface simples (por exemplo, pode ser via terminal) para interação com o usuário. Obrigatoriamente, a interface deve possuir as seguinte "telas":

1. **Tela inicial:** a interface deve apresentar duas opções ao jogador: "iniciar um novo jogo"; ou "visualizar maiores pontuações";
2. **Visualizar maiores pontuações:** a interface deve apresentar uma listagem com as 10 maiores pontuações obtidas no jogo até o momento, apenas dos jogadores que conseguiram vencer o jogo, ou seja, responder a pergunta final;
3. **Iniciar novo jogo**: caso o jogador decida iniciar um novo jogo, a interface deve inicialmente solicitar uma espécie de "nickname" ao jogador, de modo a associar o jogo que irá começar com o jogador em questão. Caso o "nickname" já tenha sido utilizado (já conste no banco de dados), o sistema deve solicitar ao jogador para indicar outro "nickname";
4. **Tela de pergunta:** após digitar seu "nickname", a interface deve prover ao jogador a primeira pergunta numa tela que deve conter, obrigatoriamente:
    a. Nível da pergunta;
    b. Ordem da pergunta (exemplo: 1 de 10);
    c. Nome do jogador;
    d. Pergunta com as 4 alternativas;
    e. Listagem de ajudas disponíveis;
    f. Pontuação obtida até o momento;

    Caso o jogador acerte, deve avançar para a próxima pergunta.

    Caso o jogador erre, deve informar uma tela de "fim de jogo" com a sua pontuação obtida. A pontuação final deve ser a obtida até a pergunta anterior (que foi corretamente respondida). Observação: Sua equipe pode implementar outra estratégia de pontuação final em caso de erro, desde que devidamente justificada no arquivo README da versão final.

    Caso o jogador solicite uma ajuda, a tela seguinte deve corresponder a ajuda solicitada;

5. **Tela de fim de jogo caso erre:** caso o jogador erre a pergunta, a interface deve mostrar a mensagem de fim de jogo com a sua pontuação obtida;
6. **Tela de "vencedor":** caso o jogador chegue até a pergunta final e acerte, a interface deve mostrar uma mensagem de parabenização, informando a sua pontuação obtida;

Cliente: Comunicação com o Servidor

O cliente deverá se comunicar com o servidor utilizando, obrigatoriamente, uma das seguintes estratégias: **RPC ou REST**. O cliente não deverá conhecer diretamente os endereços das diferentes instâncias dos servidores. Para o cliente, deverá existir apenas um ponto de acesso ao serviço: **o endereço público disponibilizado pelo ngrok** e associado ao balanceador.

As definições de comunicação entre cliente e servidor (no caso do RPC, as funções, e no caso do REST, os padrões de URL) são de responsabilidade da equipe, não havendo um requisito mínimo ou obrigatório nesta direção.

Balanceador de carga

O balanceador de carga deve operar de maneira centralizada, realizando as seguintes ações: receber uma requisição de um cliente via **ngrok**, selecionar qual é o melhor servidor dentre os 3 disponíveis, e realizar o direcionamento. Embora não seja atribuição do balanceador, para este trabalho em específico (devido às restrições de conexão e roteamento), o balanceador também deverá ser o "ponto de saída" das respostas das requisições feitas pelos clientes. A estratégia a ser implementada de balanceamento é, obrigatoriamente, a **Round-Robin (RR)** clássica.

A fins de rastreamento, para cada requisição recebida pelo balanceador, deve ser salvo em um arquivo de log uma linha com as seguintes informações:

```
IP/porta cliente; tipo requisicao; horario; servidor atribuido
```

Na qual o "tipo requisição" deve ser uma informação que identifique o que o cliente solicitou. Intuitivamente, ela deve seguir o padrão estabelecido pela equipe ao utilizar a estratégia de comunicação RPC ou REST.

Servidor

O servidor deve receber uma requisição proveniente do balanceador e realizar a operação solicitada. Para realizar as ações, o servidor deve estar conectado e realizar

consultas periódicas a um banco de dados utilizando um SGBD de conveniência da equipe. De maneira análoga, caso a requisição demande, o servidor deve realizar inclusões no banco de dados.

A estruturação da lógica de operação do servidor é de responsabilidade da equipe.

Banco de dados

O banco de dados, conjuntamente com o SGBD, terá duas atribuições principais:

- **Manter o banco de questões:** deve haver uma tabela própria para armazenar as questões que serão sorteadas para os jogadores;
- **Registrar o histórico de cada jogo**: a partir das operações realizadas por um servidor, o banco de dados deve persistir o histórico do jogo de um determinado jogador em uma tabela unificada. Toda ação realizada por um jogador dentro do jogo deve ser persistida no banco de dados. Dessa maneira, a tabela deve ter informações suficientes para indicar o estado atual do jogo, dada a requisição de um determinado jogador.

  O modo como esta tabela será elaborada é responsabilidade da equipe. Caso a equipe julgar necessário, podem ser elaboradas mais de uma tabela.

Linguagens de Programação e bibliotecas permitidas para implementação

Não há restrições de linguagens de programação a serem utilizadas neste trabalho. Também não há restrição de bibliotecas e ferramentas a serem utilizadas no desenvolvimento do SD, **exceto** para a implementação do balanceador de carga, o qual a equipe deve implementar toda a lógica operacional (por exemplo, não é permitido utilizar o NGINX).

No entanto, todas as dependências de projeto, que excedem as bibliotecas nativas da linguagem de programação a ser utilizada, devem ser devidamente descritas no arquivo de instruções de compilação e execução (descrito nas próximas seções).

Casos omissos

Quaisquer casos omissos relacionados ao desenvolvimento deste trabalho que não foram esclarecidos nesta descrição deverão ser arguidas diretamente com o professor de

maneira pública, por meio de comentários públicos na área de instruções e de submissão deste trabalho, exclusivamente via Google Classroom.

Normas para desenvolvimento e entrega do Trabalho Prático

O trabalho prático deve ser desenvolvido **por uma equipe de, no mínimo, UM aluno matriculado na disciplina e, no máximo, TRÊS alunos matriculados na disciplina.**

A submissão do trabalho prático será feita em três etapas: entrega 01; entrega 02; e entrega final. As entregas devem ser feitas exclusivamente via Classroom, na página correspondente à disciplina. **Não serão aceitos trabalhos entregues após o prazo final de cada etapa e também não serão aceitos trabalhos submetidos por outros locais (email, whatsapp, etc.)**.

Entrega 01

A primeira entrega do trabalho deve consistir em um arquivo .PDF proveniente de um conjunto de slides, o qual deve conter as definições iniciais do Quiz Online, bem como as estratégias e bibliotecas a serem utilizadas para desenvolvimento do SD. Haverá um modelo padrão de apresentação a ser utilizado pelas equipes, disponibilizado no Classroom.

Neste arquivo, as seguintes informações devem ser descritas:

- Tema do Quiz Online;
- Qual linguagem será utilizada;
- Qual estratégia de comunicação será utilizada;
- Serão utilizadas bibliotecas adicionais? Se sim, quais?
- Qual banco de dados e SGBD serão utilizados;
    - Modelagem das tabelas do banco de dados;
- Um link para documento externo com todas as perguntas que comporão o Quiz Online. Devem constar neste documento tanto as perguntas quanto as 4 alternativas, indicando qual é a correta.

Prazo de entrega: 30/09/2026 às 23:59.

Entrega 02

A segunda entrega do trabalho deverá consistir em uma primeira versão do Sistema Distribuído, contendo a implementação dos seguintes itens:

- Interface inicial do cliente implementada (tela inicial, tela de iniciar novo jogo e tela da primeira pergunta);
- Balanceador de carga operante para as solicitações das telas informadas no item anterior;
- Servidores operantes para resolver as solicitações supracitadas;
- Banco de dados operando com os servidores, com as tabelas implementadas.

É intuitivo deduzir que a comunicação entre processos dos clientes x balanceador de carga devem também estar operantes.

Os seguintes documentos devem ser entregues:

- Código fonte do ambiente de simulação desenvolvido;
- Arquivo no estilo “README” apresentando qual a versão da linguagem de programação utilizada, como compilar (se for o caso) e executar o sistema. Caso seja necessário utilizar alguma biblioteca adicional, identificar nesse arquivo.
- Uma nova versão do arquivo .PDF da Entrega 01, **apenas se houveram modificações** em algum dos itens planejados na primeira entrega.

Prazo de entrega: 08/11/2026 às 23:59.

Entrega Final

A entrega consiste no SD totalmente operante, de acordo com a especificação aqui estabelecida. Os seguintes documentos devem ser entregues:

- Código fonte do ambiente de simulação desenvolvido;
- Arquivo no estilo “README” apresentando qual a versão da linguagem de programação utilizada, como compilar (se for o caso) e executar o sistema. Caso seja necessário utilizar alguma biblioteca adicional, identificar nesse arquivo.

Prazo final: 27/11/2026 às 23:59.

Observação: O tempo estipulado para realização deste trabalho é menor do que o prazo final determinado. Mais especificamente, este trabalho possui uma duração estimada de duas semanas e meia para ser completamente desenvolvido. Dessa maneira, alunos atendidos pelo PROPAE já estão contemplados com 50% a mais do tempo necessário para desenvolvimento do trabalho.

Apresentações do Trabalho Prático (apenas para Entrega Final)

- As apresentações irão ocorrer nos dias **30 de novembro** e **03 de dezembro** de 2026, preferencialmente no horário de aula. Caso o número de equipes ultrapasse o quantitativo adequado para apresentação em horário de aula, o professor se reserva ao direito de combinar outros horários de apresentação com cada equipe.
- O cronograma de apresentação será definido unilateralmente pelo professor, sendo divulgado na véspera do primeiro dia de apresentações (ou antes, conforme disponibilidade do professor).
- O professor fornecerá 2 dispositivos computacionais para execução do sistema.
- A apresentação consistirá na demonstração de execução do sistema. Durante a demonstração, o professor se reserva ao direito de:
    - Solicitar que a equipe proceda com a execução do sistema;
    - Realizar perguntas para a equipe, ou individualmente, relacionadas à implementação do código e ao comportamento do sistema;
    - Realizar perguntas para a equipe, ou individualmente, relacionadas à fundamentação teórica vista em sala de aula, desde que esteja alinhada ao desenvolvimento do trabalho prático.

Considerações Finais

- A apresentação do trabalho é **mandatória** para **toda a equipe.**
- A performance individual e em equipe na apresentação possui **caráter de pontuação eliminatório**, ou seja: caso o professor avalie que o indivíduo, ou a equipe, não demonstram conhecimento sobre o desenvolvimento do trabalho, toda a nota pode ser zerada, mesmo que o trabalho tenha sido implementado e entregue.
- É por meio da apresentação que o professor da disciplina se **certificará de** que todos os membros da equipe contribuíram para a realização do trabalho prático!
    - **Se for utilizar ferramentas de IA, utilize com responsabilidade, e saiba aquilo que você está fazendo** 🙂